

DE Project Build

End-to-End Data Engineering Portfolio Project

■ DE PROJECTS

■ LEARNING OUTCOME

By the end of this module you will design and build a complete end-to-end data engineering project: ingestion to warehouse to dashboard, with orchestration, data quality, and a GitHub portfolio that gets you hired.

■ Skills You Will Gain

- Design a complete DE project architecture from scratch
- Build an ELT pipeline with Python + Spark + dbt + Airflow
- Set up a cloud data lake and warehouse (free tier)
- Implement data quality gates with Great Expectations
- Document the project professionally for GitHub
- Prepare for DE interview questions with this project

■ One well-documented DE project on GitHub gets more interviews than 5 certifications. Hiring managers want proof you can build things that run reliably in production.

SECTION 1

Project: NYC Taxi Analytics Pipeline

Project Overview

Dataset: NYC Taxi Trip Record Data (TLC) — millions of real trips, updated monthly Goal: Build a reliable, tested, monitored batch pipeline that transforms raw trip data into a business-ready analytics layer for a dashboard Why this project: Real GB-scale data, free to download, used by industry for benchmarks. Stack: Python + PySpark + dbt + Airflow + BigQuery (or Snowflake) + Google Looker Studio

1

Ingest

Download monthly Parquet files from TLC website → upload to GCS/S3 data lake raw zone

2

Validate

Run Great Expectations checks on raw data — schema, nulls, value ranges

3

Process

PySpark job: clean nulls, clip outliers, join zone lookup table, add derived columns

4

Silver

Write cleaned data to Silver layer as partitioned Parquet (partitionBy year, month)

5

Transform

dbt models: stg_trips → int_trips_enriched → fct_trips, dim_zones, dim_datetime

6

Test

dbt tests: not_null, unique, accepted_values, positive amounts, valid coordinates

7

Orchestrate

Airflow monthly DAG: download → validate → spark → dbt run → dbt test → notify

8

Visualise

Looker Studio / Metabase: KPI dashboard — revenue, trips, avg speed, heatmap

9

Monitor

Airflow SLA alerts, Great Expectations Data Docs, row count anomaly detection

10

Document

GitHub README with architecture diagram, results, demo link, setup steps

SECTION 2

Architecture Diagram (Text)

SOURCE DATA LAKE WAREHOUSE SERVING ===== TLC Website GCS / S3
BigQuery Dashboard (monthly Parquet) --> /raw/taxi/ --> /silver/trips --> Looker Studio year=2024/
(Spark) Metabase month=01/ /gold/ API Zone Lookup CSV --> /raw/zones/ --> fct_trips --> FastAPI
dim_zones dim_datetime (dbt models) ORCHESTRATION: Airflow DAG (monthly cron) QUALITY: Great
Expectations + dbt tests ALERTS: Email on failure, Slack on SLA miss

SECTION 3

Sample Code — Airflow DAG

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

with DAG(
    dag_id='nyc_taxi_monthly_pipeline',
    schedule_interval='0 8 2 * *', # 8 AM on 2nd of each month
    start_date=datetime(2024, 1, 1),
    catchup=True, # process all historical months
    default_args={'retries': 2, 'retry_delay': timedelta(minutes=10), 'email_on_failure': True,
                  'email': ['alerts@team.com']},
    tags=['nyc-taxi', 'monthly', 'production'],
) as dag:
    download = PythonOperator(
        task_id='download_raw_data',
        python_callable=download_taxi_data,
        op_kwargs={'year_month': '{{ ds[:7] }}'}, # e.g. '2024-01'
    )
    validate_raw = PythonOperator(
        task_id='validate_raw_data',
        python_callable=run_ge_checkpoint,
        op_kwargs={'checkpoint': 'raw_taxi_checkpoint'},
    )
    spark_process = BashOperator(
        task_id='run_spark_processing',
        bash_command='spark-submit /opt/spark/jobs/process_taxi.py {{ ds[:7] }}',
    )
    dbt_run = BashOperator(
        task_id='run_dbt_models',
        bash_command='cd /opt/dbt && dbt run --vars "{month: {{ ds[:7] }}}"',
    )
    dbt_test = BashOperator(
        task_id='run_dbt_tests',
        bash_command='cd /opt/dbt && dbt test',
    )

    download >> validate_raw >> spark_process >> dbt_run >> dbt_test
```

SECTION 4

Professional README Template

```
# NYC Taxi Analytics Pipeline > End-to-end batch pipeline processing millions of NYC taxi trips >
monthly, delivering analytics-ready data for business reporting. ## Architecture [Insert
architecture diagram image here] ## Tech Stack | Layer | Technology |
|-----|-----| | Ingestion | Python + requests | | Storage | Google Cloud
Storage| | Processing | Apache Spark (PySpark) | | Transform | dbt (BigQuery) | | Orchestration|
Apache Airflow | | Quality | Great Expectations | | Dashboard | Looker Studio | ## Key Results -
Processes 15M+ trips/month from raw to analytics-ready in under 30 min - 99.8% pipeline success
rate with automated retry and alerting - 14 dbt data quality tests enforcing business rules on
every run ## Quick Start `` git clone https://github.com/you/nyc-taxi-pipeline cp .env.example
.env # fill in GCP credentials docker-compose up -d # starts Airflow ``
```

SECTION 5

DE Interview Preparation

Question	Strong Answer Strategy
Design a pipeline for 10M events/day	Kafka → Spark Structured Streaming → Parquet on S3 partitioned by date → dbt → BigQuery
How do you handle late-arriving data?	Watermarking in Spark Streaming (10-minute grace period). Reprocess affected partitions in batch.
How do you ensure data quality?	Great Expectations tests at ingestion, dbt schema tests on every model, row count anomaly detection.
What is your dbt project structure?	Staging (clean source 1:1) → Intermediate (joins, business logic) → Marts (fact/dim tables).
How would you debug a pipeline that ran slowly today?	Check Airflow logs for task durations. Spark UI for skewed partitions or missing broadcast. C...

■ PRACTICE Q & A

- Q1. Why should you use the NYC Taxi dataset for your DE portfolio project?

A: Real GB-scale data (not toy datasets), free to download, multiple files to process, has schema changes over years (tests your handling), and interviewers recognise it as a credible real-world challenge.
- Q2. What should a DE project README always include?

A: Architecture diagram, tech stack table, key results with numbers (rows processed, latency, success rate), how to reproduce locally (docker-compose up), and links to live demo or dashboard if possible.
- Q3. How do you demonstrate data quality in a portfolio project?

A: Show Great Expectations HTML data docs, dbt test results in CI, and a dbt source freshness check. Explain what each test validates and what business rule it enforces.

Q4. What makes a DE project portfolio-worthy vs just a tutorial?

A: Real data, production patterns (retry logic, alerting, idempotency, monitoring), clear documentation, and a result you can describe quantitatively: 'processes X rows in Y minutes with Z% reliability'.

Q5. What three DE interview questions should you always prepare answers for?

A: 1) Design a pipeline for [scale] — use: source → Kafka/batch → lake → warehouse → BI. 2) How do you handle failures — use: retry, idempotency, alerting, DLQ. 3) How do you ensure data quality — use: GE, dbt tests, monitoring.

■ DE Project Cheat Sheet	
Bronze Layer	Raw ingested data – exact copy of source
Silver Layer	Cleaned, validated, deduplicated
Gold Layer	Business-ready fact and dim tables
dbt run + dbt test	Transform + validate in warehouse
Airflow DAG	Schedule and orchestrate pipeline
Great Expectations	Data quality validation framework
Parquet + partitioned	Efficient cloud storage format
docker-compose up	Start Airflow locally for development
terraform apply	Provision cloud infra as code
Medallion architecture	Bronze → Silver → Gold pattern
SLA monitoring	Alert if pipeline misses deadline
README with numbers	X rows, Y minutes, Z% reliability